

COMP2221 Systems Programming Summative Coursework Report



1 SUMMARY OF APPROACH & SOLUTION DESIGN 1.2 Diagrams

1.1 Definition of Solution

This solution implements sequential, 40-byte aligned blocks containing header, payload, and footer. The design prioritises robustness for critical missions in "Mars's unforgiving environment".

Structure:

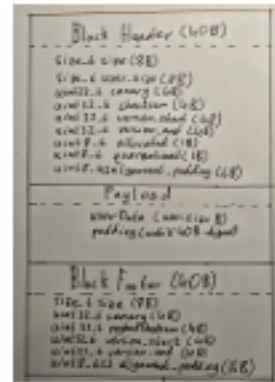
Blocks contain 40-byte-aligned headers, payloads, and footers. Headers store size (total block size), `user_size` (user-allocated bytes), `canary` (fast header validation), `checksum` (metadata validation), `version_start/end` (brownout detection), `allocated`, and `quarantined` (booleans). Footers store size (mirrored), `canary` (fast footer validation), and `payloadChecksum` (payload validation). Fixed 40-byte chunks ensure payload addresses are aligned, causing significant overhead for small allocations (~93% for 8B), but less for large allocations (~7% for 1KB). Storing `user_size` denies access beyond allocated space (`mm_read`, `mm_write`).

Corruption Detection:

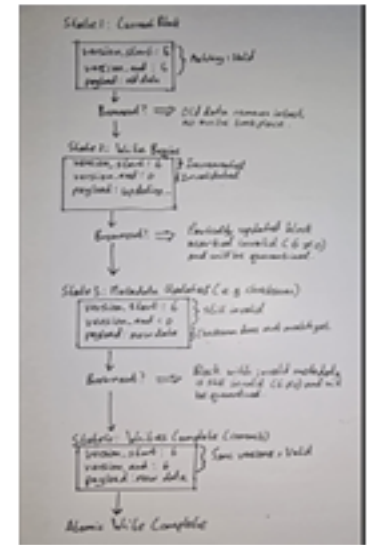
Canaries enable fast detection of obvious corruption [1]. Checksums validate metadata (`generateChecksum`) and payload (`generatePayloadChecksum`) at single-bit levels [2]. Version Protocol provides atomic brownout protection. Metadata changes increment `version_start`, set `version_end=0`, perform writes with immediate verification (`createBlock`), then update `version_end`. Differing versions indicate brownout induced partial writes. Footers duplicate size for validation since it's critical for heap traversal. These aspects are all validated by the `isValid` helper function on block access. `mm_init` validates the 5-byte free-pattern.

Functionality:

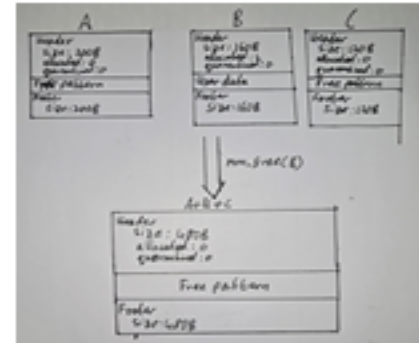
`mm_malloc` uses first-fit search through implicit free-list, splitting blocks when surplus space is large enough for a minimal block (120B). `mm_free` writes the 5-byte free-pattern, marks block unallocated, then coalesces adjacent free-blocks (`coalesce`). Footers enable $O(1)$ backward traversal (`get_prev_block`), avoiding $O(n)$ heap traversal. `createBlock` standardises metadata updates with brownout protection. `mm_read` validates block integrity and restricts length from offset to `user_size`. `mm_write` enforces `offset+len==user_size` ensuring writes fill the allocation space exactly.



(a) Allocated block structure



(b) Atomic write brownout protection



(c) `mm_free` process with coalesce

Fig. 1: Block structure, atomic write protocol, and free-block coalescing

2 ANALYSIS OF SOLUTION

2.1 Memory

My design makes trade-offs on space efficiency for robustness when necessary. The 40-byte alignment alongside the header/footer structure causes per-block overhead of 80-119B, especially apparent for smaller block allocations.

Allocation	Block Size	User Data as % of Payload	Overhead
8B	120B	20%	~93%
50B	160B	~63%	~69%
100B	200B	~83%	50%
500B	600B	~96%	~17%
1000B	1080B	100%	~7.4%
5000B	5080	100%	~1.6%

TABLE 1: Per-Block User Data & Overhead

Mars Rovers have historically had 0.5-256MB RAM [3], mitigating the overhead's significance. Allocations of 1KB+ hold sub-10% overhead. This is acceptable considering the level of environmental memory interference on Mars and the subsequently necessary robust corruption detection mechanisms. Variable header padding could reduce overhead compared to fixed 40-byte padding, but would require more complex implementation and make heap-structure more susceptible to irrecoverable corruption if padding data was affected.

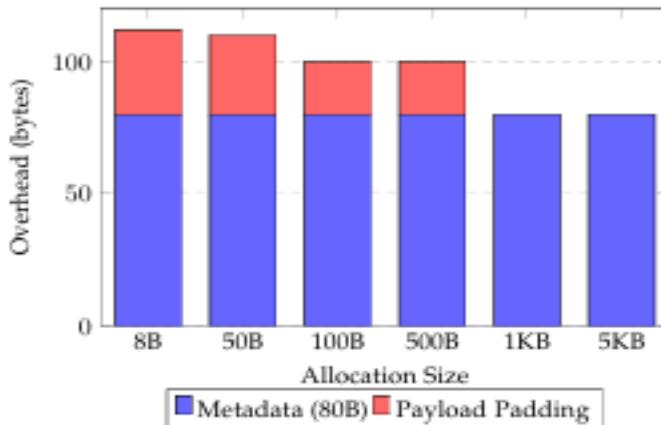


Fig. 2: Overhead breakdown

Coalescing adjacent blocks mitigates fragmentation, however payload alignment causes internal fragmentation with space lost to payload padding.

2.2 Runtime

`mm_malloc` has $O(n)$ worst-case complexity, but is typically sub-linear in practice since first-fit search often finds blocks earlier in the implicit free-list. Implementing an external linked free-list [4] would reduce search time, but increase memory overhead and corruption potential. Best-fit would reduce fragmentation [5], but require scanning the entire

heap, making every allocation $O(n)$.

`mm_free` has $O(p)$ runtime as `coalesce` uses headers/footers for $O(1)$ forwards/backward traversal [6], and writing the free pattern across the payload (length p) causes $O(p)$ runtime.

Function / Approach	Best Case	Average Case	Worst Case
<code>mm_malloc</code> First-fit search (n sized heap)	$O(1)$	$O(n/2)$	$O(n)$
<code>mm_free</code> Bidirectional coalescing + write p-length free-pattern	$O(p)$	$O(p)$	$O(p)$
<code>mm_read</code> Copy m bytes to buffer	$O(m)$	$O(m)$	$O(m)$
<code>mm_write</code> Copy m bytes + CRC-32 checksum calculation	$O(m)$	$O(m)$	$O(m)$
<code>mm_realloc</code> (in-place) Expand user_size in-place	$O(1)$	$O(1)$	$O(1)$
<code>mm_realloc</code> (new block) <code>mm_malloc</code> + copy m bytes	$O(1+m)$	$O(n/2 + m)$	$O(n + m)$

TABLE 2: Function Time Complexity

CRC-32 checksums [7] (`generatePayloadChecksum`) require each bit in the user-data to be processed, with $O(\text{user_size})$ complexity. Per-byte XOR checksums would reduce runtime overhead, but at the cost of less granular error-detection. Called on every `mm_write` and `isValid`. The Computational cost for bit-level corruption detection is necessary for radiation maintaining consistent data on Mars.

Target Data	Size	CRC-32 Ops	XOR Ops	CRC-32 Overhead
Metadata	22B	176	22	+154 Ops (8x)
User Data (100B)	100B	800	100	+700 Ops (8x)
User Data (1KB)	1024B	8192	1024	+7168 Ops (8x)

TABLE 3: Checksum Costs

3 USE OF GENERATIVE AI, TOOLS, OR OTHER RESOURCES

CRC-32 Implementation:

My implementation of the standard CRC-32 algorithm was supported by Gen AI, like explaining which CRC-polynomial to use and what the underlying purpose was. This helped me understand the algorithm conceptually and translate it to C code accurately. This helped ensure correctness and deepened my understanding compared to implementing from example code.

Brownout Detection Suggestions: After putting brownout detection mechanisms in key areas like `mm_write` and `createBlock`, Gen AI was useful to discuss other possible impacts of brownouts throughout the program and where it would be valuable to add extra validation. This sped up what would've been more trial and error to ensure that detection was thorough enough to pass further testing I didn't yet have access to. In this context, the Gen AI hindered me when it began suggesting redundant data validations which added no additional corruption detection but added unnecessary operations to each function call.

Automated Testing:

Gen AI helped design the `runme.c` testing file clarifying core aspects of the design which should be verified with example use-cases, including boundary enforcement testing (Test 3) and thread safety validation (Test 10).

Syntax Linting:

Google code style linting was checked using `cpplint` [8] python file. This significantly sped up the process of syntax formatting.

4 ADDITIONAL FUNCTIONALITY

4.1 Definition of Additional Functionality

Memory Reallocation:

`mm_realloc` follows 3 main paths.

For allocation increases fitting within the blocks payload, the `user_size` is expanded in-place, overwriting padding.

For allocation decreases, the `user_size` is reduced in-place, replacing deallocated space with padding. If there is sufficient surplus space, a new block is split-off.

For allocation increases that don't fit within the payload, the next block is expanded into if unallocated. Surplus from that block is also split off.

If these 3 paths are not possible, it defaults to `mm_malloc` [9], allocating a new block, copying the data, and freeing the original block. Copying the data is brownout protected by comparing the bytes written to the initial `user_size`, cross-referencing each byte, and an `isValid` check. Any inconsistency causes the new block to be freed and NULL to be returned, preserving original user data.

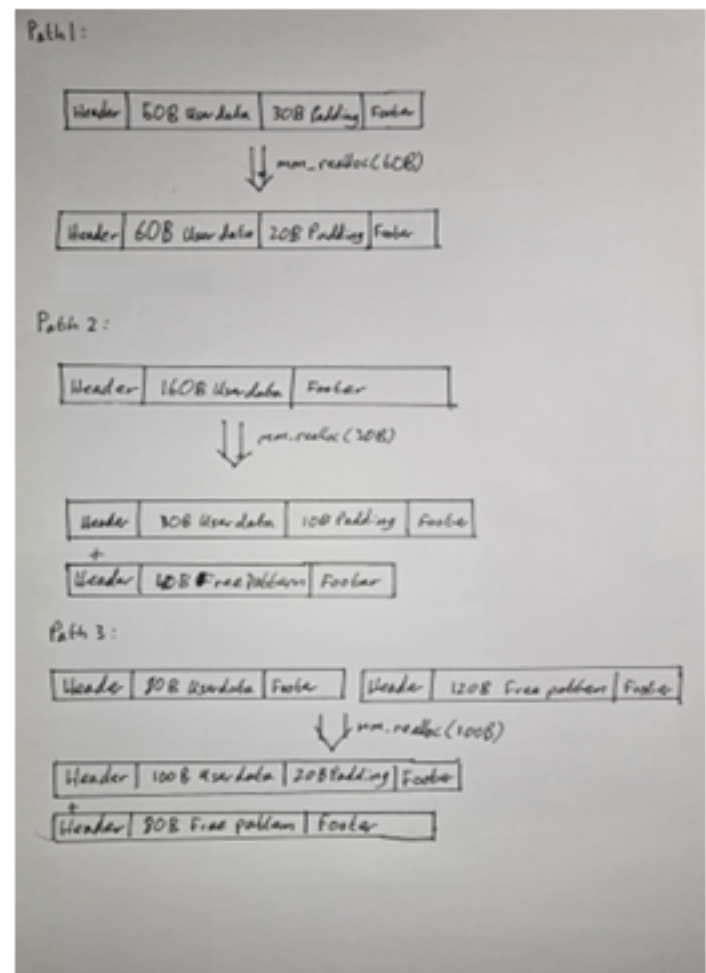
Implementing backwards expansion would've further optimised space utilisation and mitigated fragmentation, however would involve complex multi-block and data copying presenting severe brownout risk.

Thread Safety:

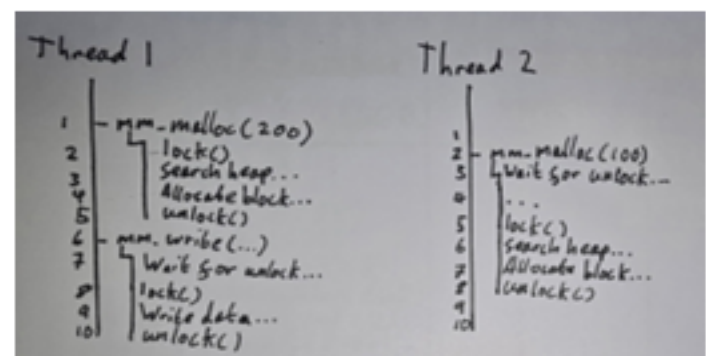
`pthread_mutex` [10] implements coarse-grained locking, protecting the heap from concurrent operations. The global mutex restricts heap access to one thread at a time, acquired at each function call and released when returning. In `mm_realloc` the mutex is unlocked before calling `mm_malloc/mm_free`, avoiding deadlocks.

Coarse-grained locking is a simple method of blocking threads from accessing the heap, most appropriate for systems with limited concurrency. Concurrent access to non-conflicting areas of the heap is blocked, potentially impacting performance. Per-block locks could permit more concurrent access, but would add memory overhead and complexity.

4.2 Diagrams



(a) Reallocation Pathways



(b) Thread Safe Architecture

Fig. 3: Reallocation Strategies and Thread Safety

REFERENCES

- [1] E. Heymann and B. P. Miller, "Memory safety checks (chapter 27)," October 2025. [Online]. Available: <https://research.cs.wisc.edu/mist/SoftwareSecurityCourse/Chapters/27-MemorySafetyChecks.pdf>
- [2] C. Borchert, H. Schirmeier, and O. Spinczyk, "Compiler-implemented differential checksums: Effective detection and correction of transient and permanent memory errors," in *2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2023, pp. 81–94.
- [3] "Comparison of embedded computer systems on board the mars rovers," Aug 2025.
- [4] G. Bill, "Memory allocation strategies - part 5," Nov 2021. [Online]. Available: <https://www.gingerbill.org/article/2021/11/30/memory-allocation-strategies-005/>
- [5] Sunshine, Sep 2019. [Online]. Available: <https://www.sunshine2k.de/articles/coding/cmемalloc/cmемory.html>
- [6] A. Bracy, "Dynamic memory allocation," 2016. [Online]. Available: <https://www.cs.cornell.edu/courses/cs3410/2016sp/slides/16-allocation.pdf>
- [7] Bewing, "Crc32 - osdev wiki," 2009. [Online]. Available: <https://wiki.osdev.org/CRC32>
- [8] "C++ style guide." [Online]. Available: <https://google.github.io/styleguide/cppguide.html>
- [9] "Realloc." [Online]. Available: <https://en.cppreference.com/w/c/memory/realloc>
- [10] D. A. Levine, *POSIX programmer's guide: Writing portable unix programs with the POSIX. 1 standard*. O'Reilly & Associates, 1994.